BUSINESS INTELLIGENCE AND DATA ANALYTICS
– WE PREDICT THE FUTURE –

University of Petra

307401
Big Data and Data Warehouses

Introduction to Hadoop

Types of Processing in Computing

Processing Type	Description	Example
Sequential Processing	Tasks are executed one after another in a linear sequence.	Reading and executing lines of code in a single-threaded program.
Multi-Tasking	Multiple tasks share the same CPU by switching between them rapidly.	Running multiple applications (browser, music player, and document editor) on a computer.
Parallel Processing	Multiple processors or cores execute different parts of a task simultaneously.	Image processing using multiple CPU cores.
Distributed Processing	Tasks are divided across multiple computers working together over a network.	Cloud computing and blockchain networks.

Speedup Concept

Formula:

$$Speedup(S) = \frac{T_1}{T_p}$$

Where:

- T_1 = Execution time with a single processor.
- T_p = Execution time with p processors.

Key Points:

- **Definition:** Speedup measures the performance gain achieved by using multiple processors compared to a single one.
- **Linear Speedup:** Ideal scenario where $S = p$ (e.g., 4 processors give 4× speedup).
- **Superlinear Speedup:** Occasionally, $S > p$ due to better memory utilization or caching effects.
- **Sublinear Speedup:** Speedup is less than the number of processors due to communication and synchronization overhead.
- **Amdahl's Law:** The speedup is limited by the sequential portion of the program:

$$S = \frac{1}{(1-P) + \frac{P}{p}}$$

- **Gustafson's Law:** Speedup increases with problem size, making large-scale distributed computing more efficient:

$$S = p - (1-P) \times (p-1)$$

Introduction to Hadoop

- Hadoop is an open-source framework overseen by Apache Software Foundation which is written in Java for storing and processing of huge datasets with the cluster of commodity hardware.
- There are mainly two problems with the big data that Hadoop addresses:
  1. The need to store huge amount of data.
  2. The need to process that stored data.
- The traditional approach like RDBMS is not sufficient due to the heterogeneity of the data.
- Hadoop serve as the solution to the problem of big data i.e. storing and processing big data.

History of Hadoop

Doug Cutting

Mike Cafarella

- Early Beginnings (2002-2003):
  - Nutch Project: Open-source search engine project by Doug Cutting and Mike Cafarella.
  - Challenges in managing and processing large volumes of web data.
- Google's Influence (2003-2004):
  - Google Papers: Introduction of Google's GFS (Google File System) and MapReduce.
  - Inspiration for Hadoop's core concepts.
- Birth of Hadoop (2005-2006):
  - Creation of Hadoop as a separate project within Apache Nutch.
  - Named after Doug Cutting's son's toy elephant.
- Yahoo! began investing in and contributing to Hadoop's development.
- Apache Hadoop (2008):
  - Hadoop became an Apache top-level project.
  - Core components: HDFS (Hadoop Distributed File System) and MapReduce.
  - Rapid growth in community contributions and adoption.
- Ecosystem Expansion (2009-2011):
  - New projects: Hive, Pig, HBase, Zookeeper.
  - Formation of companies like Cloudera for commercial support.
- Industry Adoption (2012-2014):
  - Mainstream use by major companies: Facebook, LinkedIn, eBay.
  - Introduction of Hadoop 2.0 with YARN (Yet Another Resource Negotiator).
  - Hadoop Summit: Annual conference for advancements and use cases.
- Modern Era (2015-Present):
  - Widespread enterprise adoption for various Big Data applications.
  - Integration with cloud platforms: AWS EMR, Google Cloud Dataproc.
  - Ongoing development and enhancements to the Hadoop ecosystem.
- Impact of Hadoop:
  - Pioneered the Big Data revolution.
  - Enabled scalable, distributed data processing.
  - Fostered a strong open-source community and ecosystem.

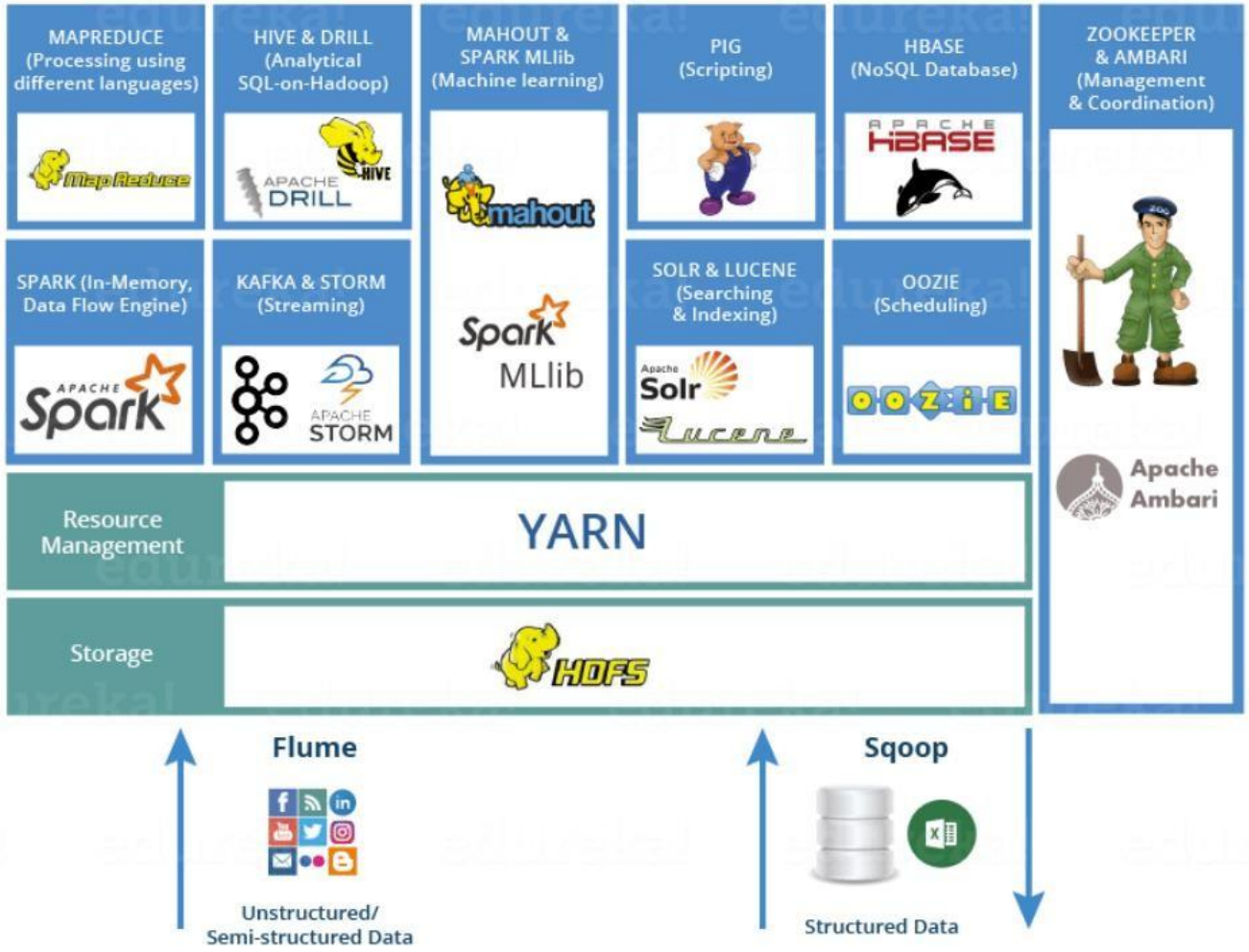
Hadoop Echo System

- Hadoop ecosystem is a suite of tools and frameworks

that enables Hadoop to handle large amounts of data. Basically, Hadoop has the following three components:

1. HDFS (Hadoop Distributed File System): A distributed file system that provides high-throughput access to application data.
2. MapReduce: A framework for parallel processing of large data sets.
3. YARN, Resource Manager.

Other ecosystem components: Hive, Pig, HBase, Storm, Spark, Kafka, Flume, Sqoop, Oozie, Zookeeper…etc. These components **work together** to provide a comprehensive solution for managing and processing large data sets in a distributed computing environment.



HDFS – Hadoop Distributed File System

What is HDFS?

- **Hadoop Distributed File System (HDFS)** is the storage layer in Hadoop.
- It enables **distributed storage** and **fault tolerance** by replicating data across multiple nodes.

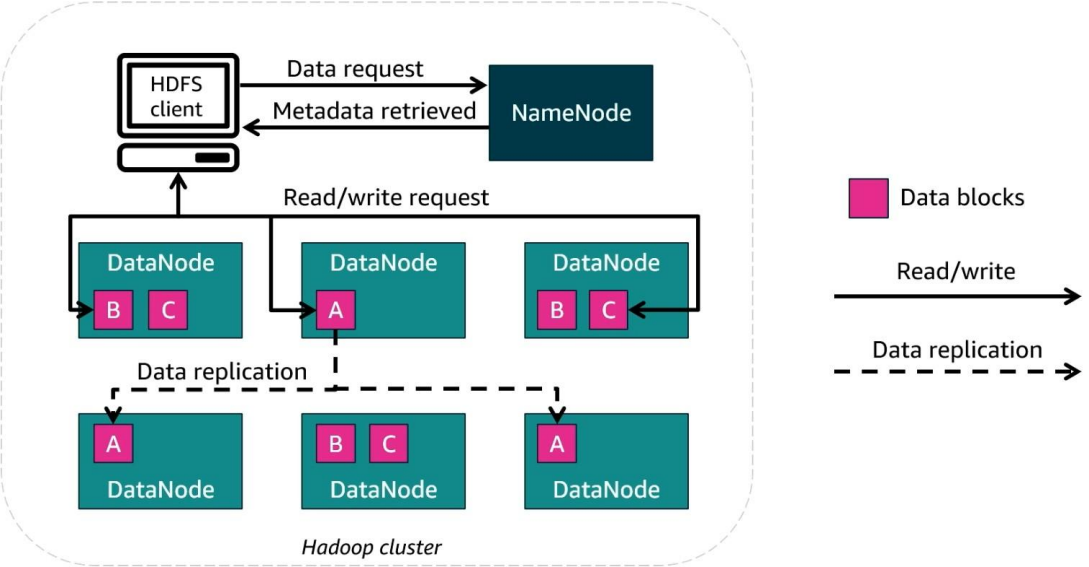
- **Key Components of HDFS**

1. NameNode

- Acts as the **master node**.
- Manages **metadata** (file locations, permissions, and structure).
- Does not store actual data.

2. DataNodes

- Store actual data in **blocks**.
- Handle read/write requests from clients.
- Perform **data replication** to ensure fault tolerance.



How HDFS Works?

1. **Client requests data** from HDFS.
2. **NameNode retrieves metadata** and provides block locations.
3. **Client reads/writes data blocks** directly from/to the DataNodes.
4. **Data replication** occurs across multiple DataNodes for redundancy.

YARN, Yet Another Resource Negotiator

What is YARN?

- YARN (Yet Another Resource Negotiator) is the resource management layer in Hadoop.
- It efficiently allocates resources and schedules tasks for processing big data jobs.

Key Components of YARN

1. Resource Manager (RM) - Central component managing resource allocation.

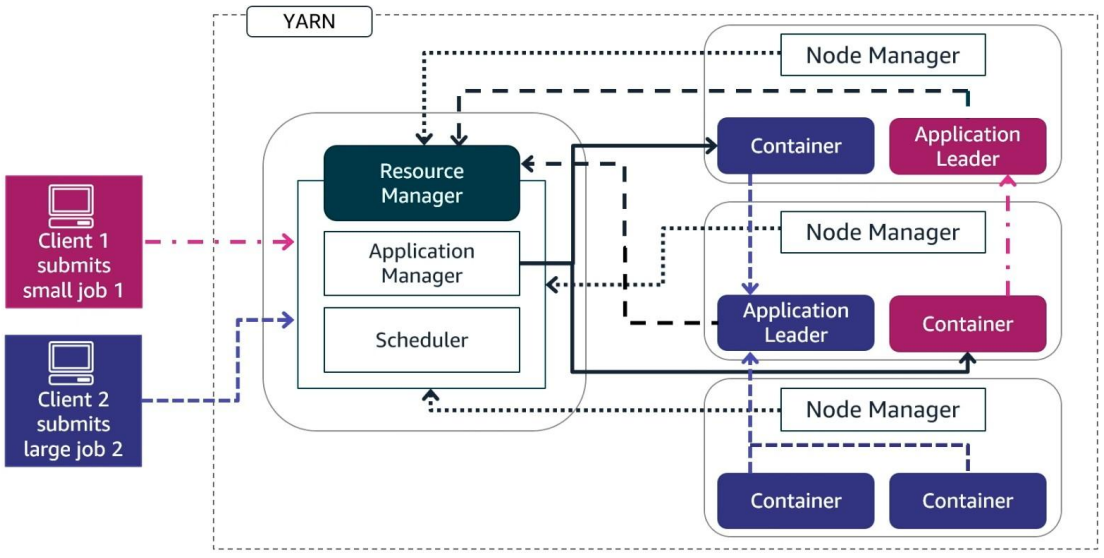
- It has two main parts:
  - **Application Manager** – Manages job submission and execution.
  - **Scheduler** – Allocates resources to tasks based on policies.

2. Node Managers (NM)

- Runs on each DataNode to execute tasks.
- Manages **Containers**, which are allocated resources for executing tasks.

3. Application Master (AM)

- Runs within a container and coordinates job execution.
- Communicates with the Resource Manager and Node Managers.

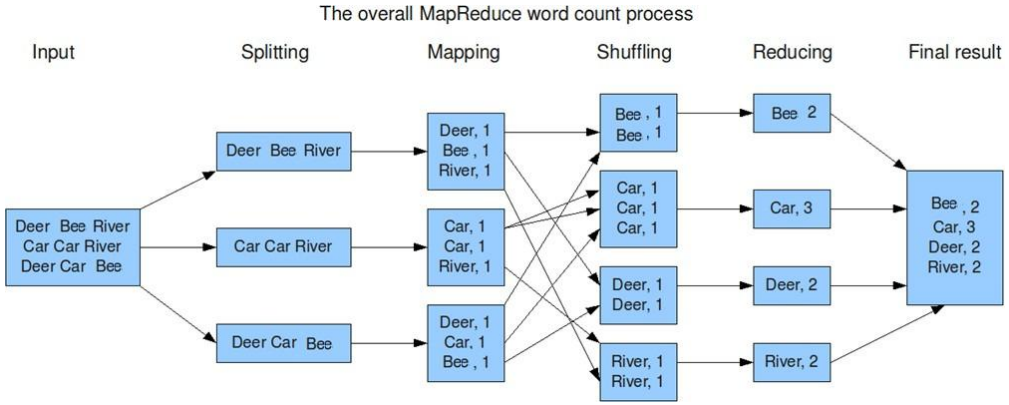


How YARN Works?

1. **Clients submit jobs** to the Resource Manager.
2. **Scheduler assigns resources** based on job size and priority.
3. **Application Master is launched** in a container to oversee execution.
4. **Node Managers execute tasks** in their respective containers.
5. **Results are sent back to clients** after completion.

MapReduce

- Map Reduce is a programming model for processing large data
- sets with a parallel, distributed algorithm on a cluster.
- Key Components:
  - Mapper: Takes a set of data and converts it into another set of data, where individual elements are broken down into key/value pairs.
  - Reducer: Takes the output from a mapper as an input and combines those data tuples into a smaller set of tuples.



- Workflow:
  - Input Splits: The input file is split into multiple pieces.
  - Map Phase: Processes the input splits and produces intermediate key-value pairs.
  - Shuffle and Sort Phase: Organizes the intermediate data by key.
  - Reduce Phase: Aggregates the sorted key-value pairs and generates the final output.
- Examples:
  - Word Count: Count the number of occurrences of each word in a text file.
  - Log Analysis: Process web server logs to find the number of hits per URL.

Workflow Scenario from the Client to NameNode/DataNode with Internals and Subcomponent

1. Client Submission:

- The client submits a MapReduce job to the Hadoop cluster. The job includes the Mapper and Reducer code, and configuration parameters. The data is already stored in the Hadoop Distributed File System (HDFS)

2. ResourceManager and Job Scheduling:

- The ResourceManager receives the job submission and delegates the job to the ApplicationsManager.
- The ApplicationsManager allocates a new ApplicationMaster (AppMaster)
- instance for the job.

3. ApplicationMaster Initialization:

- The ApplicationMaster negotiates resources from the ResourceManager and requests containers (execution environments) on various NodeManagers (Data Nodes).

4. Data Splitting:

- The ApplicationMaster communicates with the NameNode to locate the input data blocks stored across the DataNodes.
- The input data is split into chunks, and the splits are assigned to the containers on the DataNodes where the data is located.

5. Mapper Execution:

- The NodeManagers launch Mapper tasks in the allocated containers.
- Each Mapper processes its assigned data chunk, producing intermediate key-value pairs.

6. Shuffling and Sorting:

The intermediate key-value pairs are shuffled and sorted by key. This involves transferring data across the network to group all values for the same key together.

7. Reducer Execution:

The NodeManagers launch Reducer tasks in containers.

Each Reducer processes its assigned key group, aggregating the values to produce the final output.

8. Writing the Output:

The Reducers write the final output to HDFS, which is managed by the NameNode.

The ApplicationMaster monitors the job's progress and updates the ResourceManager.

9. Job Completion:

Once all Mapper and Reducer tasks are complete, the ApplicationMaster informs the ResourceManager.

The client is notified of the job's completion, and the final results are available in HDFS

Map Reduce Movie Count Example

Writing the Mapper

USER ID	MOVIE ID	RATING	TIMESTAMP
196	242	3	881250949
186	302	3	891717742
196	377	1	878887116
244	51	2	880606923
166	346	1	886397596
186	474	4	884182806
186	265	2	881171488

Map →

3,1
3,1
1,1
2,1
1,1
4,1
2,1

Shuffle & Sort →

1 -> 1, 1
2 -> 1, 1
3 -> 1, 1
4 -> 1

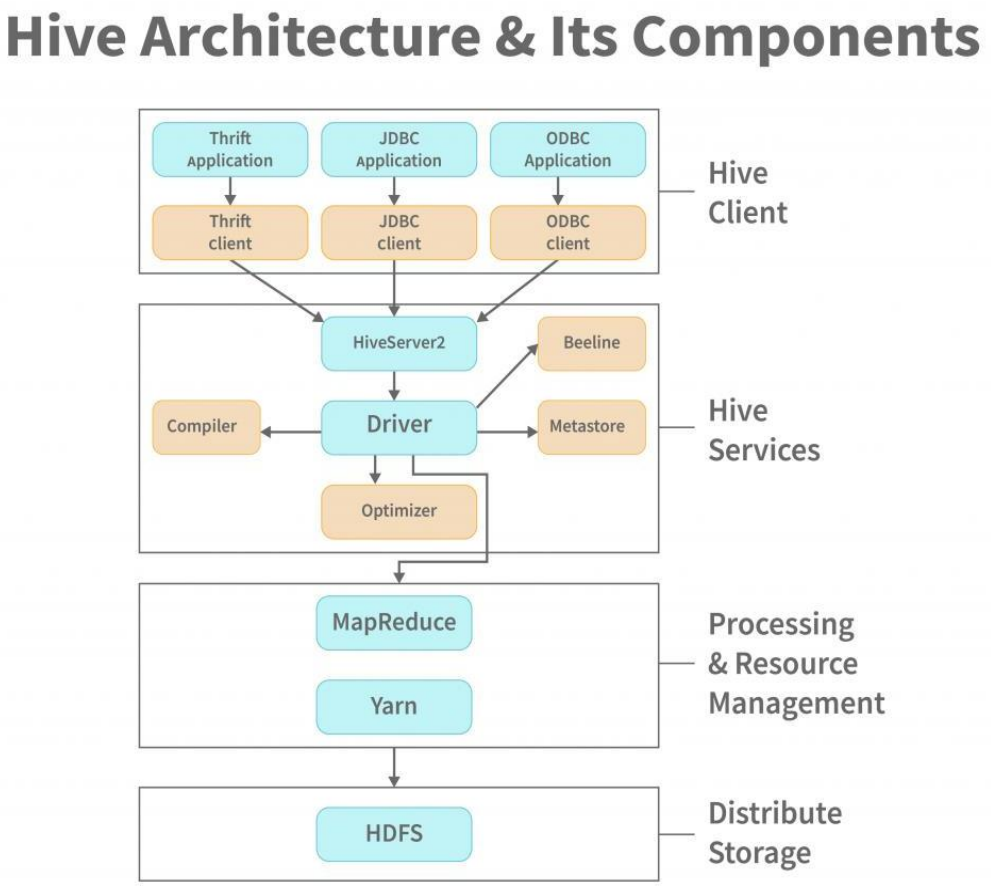
Reduce →

1, 2
2, 2
3, 2
4, 1

```python
from mrjob.job import MRJob
from mrjob.step import MRStep

class RatingsBreakdown(MRJob):

    def steps(self):
        return [
            MRStep(mapper=self.mapper_get_ratings,
                   reducer=self.reducer_count_ratings)
        ]

    def mapper_get_ratings(self, _, line):
        (userID, movieID, rating, timestamp) = line.split('\t')
        yield rating, 1

    def reducer_count_ratings(self, key, values):
        yield key, sum(values)

if __name__ == '__main__':
    RatingsBreakdown.run()
```

Hive-Data Warehouse on Top of Hadoop

- **Hive**: A data warehouse infrastructure built on top of Hadoop f using SQL-like syntax (HiveQL).
- Create by Facebook to solve MapReduce challenges.
- **Components**:
  - **Metastore**: Stores metadata for Hive tables.
  - **Driver**: Manages the execution of HiveQL statements.
  - **Compiler**: Converts HiveQL queries into MapReduce tasks.
- **Use Cases**: Data summarization, query, and analysis of
- data in HDFS.



Sample Hive Commands – Declarative, SQL Like Commands

Benefits of Hive over MapReduce

1. **SQL-Like Query Language**: HiveQL is similar to SQL, making it easy for people with SQL knowledge to work with big data.
2. **Schema on Read**: Hive defines the schema when the data is read, not when it's loaded, which is ideal for unstructured or semi-structured data.
3. **Data Summarization and Analysis**: Hive is optimized for data warehousing tasks, such as summarization, analysis, and ad-hoc querying.
4. **Ease of Integration**: Hive works seamlessly with Hadoop's HDFS, making it efficient for large-scale data processing.

Small Hive Query Example: Word Count

Let's demonstrate a simple word count using Hive. We'll load a text file and count the occurrences of each word, much like we did with Pig.

1. Create a Table

Assume we have a text file with a line of words. First, we create an external table pointing to this file.

sql Copy code

```sql
CREATE EXTERNAL TABLE IF NOT EXISTS words (
  line STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\n'
LOCATION '/path/to/textfile/';
```

2. Transform and Count Words

This next part uses Hive's built-in functions to split lines into words and count their occurrences.

sql Copy code

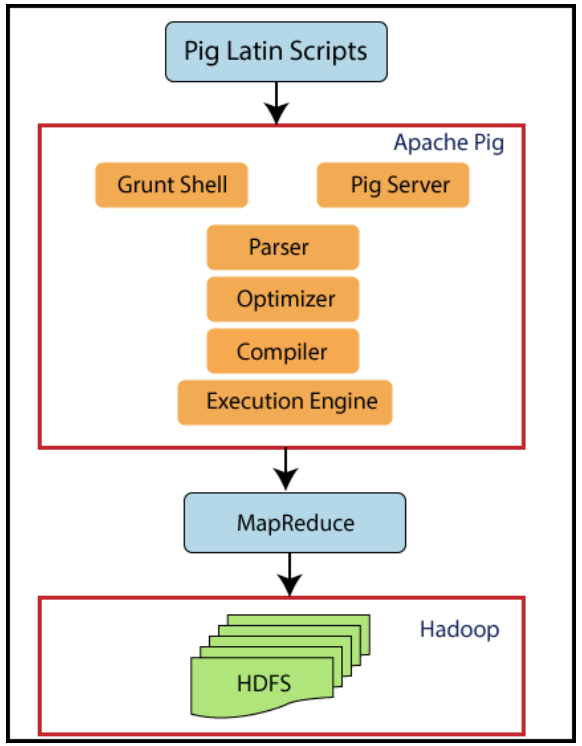
```sql
-- Split each line into words
CREATE TABLE words_split AS
SELECT explode(split(line, ' ')) AS word
FROM words;

-- Count occurrences of each word
SELECT word, COUNT(*) AS count
FROM words_split
WHERE word != ''
GROUP BY word
ORDER BY count DESC;
```

https://youtu.be/LEzQVPvNTNo?si=JQlcwb1fimzxVs_9&t=753

Apache PIG – High Level SQL Like Language on top of Hadoop

APACHE PIG

- Pig Latin is a high-level data processing language that simplifies writing
- data transformations and analysis tasks over large datasets in Hadoop.
- It's part of the Apache Pig platform, which runs on Hadoop and translates
- Pig Latin scripts into MapReduce jobs.
- This allows users to work with big data using simpler syntax compared to
- writing complex MapReduce code directly.
- Created by Yahoo to solve MapReduce challenges.
- **Architecture**:
  - **Pig Latin Scripts**: Define data transformation tasks.
  - **Pig Engine**: Executes these scripts on Hadoop.
- **Use Cases**: Data cleansing, ETL (Extract, Transform, Load) operations.



https://www.analyticsvidhya.com/blog/2022/06/getting-started-with-apache-pig/

Hive vs Pig

Sample Pig Programming Language Commands

- **Benefits of Pig Latin over MapReduce**
- **Simpler Syntax**: Pig Latin is more concise and easier to understand, making it accessible to those with SQL knowledge but without a deep understanding of Java.
- **Less Code**: A Pig Latin script requires fewer lines of code compared to MapReduce, reducing development time.
- **Built-in Operations**: Pig provides built-in operators for common data operations like filtering, grouping, and joining, which would require more extensive coding in MapReduce.
- **Data Flow Language**: Pig Latin is designed to handle data transformations in a flow-like manner, which is simpler than breaking down tasks into the Map and Reduce steps required by MapReduce.

Sample Pig Latin Script: Word Count

This script reads a text file, splits each line into words, and counts the occurrences of each word—a classic example in data processing.

pig

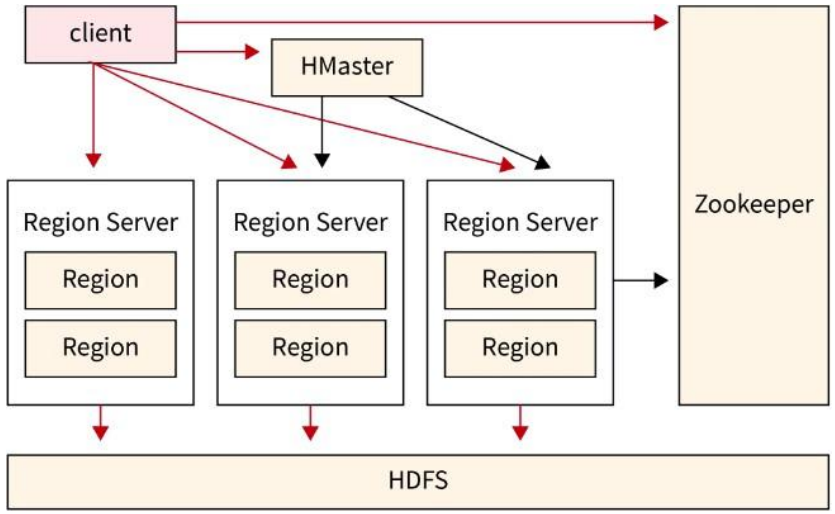
Copy code

```
-- Load a text file
data = LOAD 'sample.txt' AS (line:chararray);

-- Split each line into words
words = FOREACH data GENERATE FLATTEN(TOKENIZE(line)) AS word;

-- Filter out empty words
filtered_words = FILTER words BY word != '';

-- Group by each word
grouped_words = GROUP filtered_words BY word;

-- Count occurrences of each word
word_count = FOREACH grouped_words GENERATE group AS word, COUNT(filtered_words) AS count;

-- Display the result
DUMP word_count;
```

https://youtu.be/LEzQVPvNTNo?si=JS4-3Qxyu3o7TD9q&t=873

Apache HBase - Distributed Database on top of Hadoop

- HBase is a NoSQL database built on top of Hadoop, specifically designed to handle large amounts of sparse, semi-structured, and unstructured data in a distributed environment.
- It provides random, real-time read and write access to data, making it well-suited for big data applications where fast access to individual rows of data is essential.
- **Architecture**:
  - **HMaster**: Manages the cluster.
  - **RegionServer**: Handles read/write requests for regions of the
  - database.
  - **ZooKeeper**: Manages distributed coordination.
- **Use Cases**: Real-time read/write access to big data, suitable for large-scale storage



Apache HBase Architecture

https://www.scaler.com/topics/hadoop/hbase-architecture/

Key Features of HBase

- **Column-Oriented Storage**: HBase stores data in a column-family format, making it efficient for querying specific columns in massive datasets.
- **Real-Time Read/Write**: Unlike Hive, which is optimized for batch processing, HBase supports real-time CRUD (Create, Read, Update, Delete) operations.
- **Scalability**: HBase can scale horizontally by adding more nodes to the cluster, handling petabytes of data.
- **Strong Consistency**: Ensures that all data reads reflect the most recent writes, which is vital for applications requiring accurate, up-to- date information.
- **Integration with Hadoop**: HBase stores data in HDFS, meaning it can leverage the fault- tolerance and scalability features of Hadoop.

Basic HBase Example: Storing and Retrieving Data

Let's demonstrate some basic operations in HBase using HBase Shell commands. In this example, we'll create a table, insert data, and retrieve it.

Step 1: Create a Table

shell Copy code

```shell
# Creating a table with two column families: info and stats
create 'users', 'info', 'stats'
```

Step 2: Insert Data

shell Copy code

```shell
# Adding user information
put 'users', 'user1', 'info:name', 'Alice'
put 'users', 'user1', 'info:email', 'alice@example.com'
put 'users', 'user1', 'stats:logins', '5'

put 'users', 'user2', 'info:name', 'Bob'
put 'users', 'user2', 'info:email', 'bob@example.com'
put 'users', 'user2', 'stats:logins', '3'
```

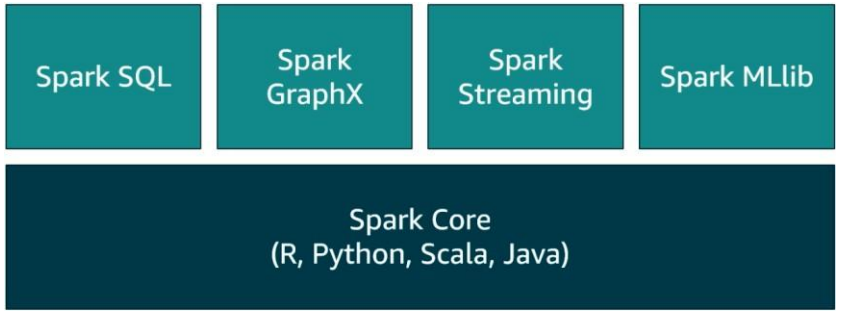
Step 3: Retrieve Data

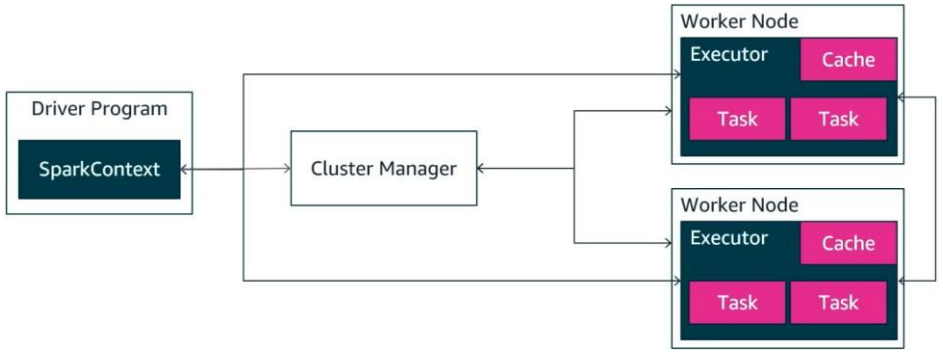
shell Copy code

```shell
# Get all data for a specific user
get 'users', 'user1'

# Retrieve specific columns
get 'users', 'user1', {COLUMN => 'info:name'}
```

Apache Spark

- Apache Spark is an open-source distributed data processing engine optimized for large-scale data processing.
- **In-Memory Processing**: Spark keeps data in memory between transformations, speeding up processing times significantly compared to disk-based MapReduce.
- Unlike Hadoop's MapReduce, Spark is designed for speed and ease of use, handling both batch and real-time processing.
- Spark offers APIs for various languages (Java, Scala, Python, R) and includes libraries for SQL, machine learning, streaming, and graph processing, making it versatile for big data and machine learning tasks.
- Core Components:
  1. **Spark Core:** The foundation for Spark's API, handling basic data
     - processing and distributing tasks across the cluster.
  2. **Spark SQL:** Enables SQL-like queries on data stored in distributed datasets (DataFrames and Datasets).
  3. **Spark Streaming:** Allows processing of real-time data streams, enabling near real-time analytics.
  4. **MLlib:** Spark's machine learning library, offering tools for scalable
     - machine learning.
  5. **GraphX:** A graph processing library for large-scale graph computations.
- **Use Cases**: Batch processing, real-time data streams, machine learning, and
- graph processing.





Basic Spark Example: Word Count in PySpark

This example demonstrates the classic word count problem, showing how easy it is to implement in Spark.

python Copy code

```python
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("WordCount").getOrCreate()

# Load text file into RDD
text_file = spark.sparkContext.textFile("path/to/textfile.txt")

# Perform word count
word_counts = (text_file
               .flatMap(lambda line: line.split(" "))  # Split lines into words
               .filter(lambda word: word != "")  # Filter out empty words
               .map(lambda word: (word, 1))  # Map each word to a count of 1
               .reduceByKey(lambda a, b: a + b))  # Sum counts for each word

# Collect and print results
for word, count in word_counts.collect():
    print(f"{word}: {count}")

# Stop Spark session
spark.stop()
```

Explanation

1. **SparkSession**: The entry point for working with Spark, used to initialize the Spark context.
2. **textFile**: Loads a text file into an RDD (Resilient Distributed Dataset), Spark's distributed collection of data.
3. **flatMap**: Splits each line into words.
4. **filter**: Removes any empty strings.
5. **map**: Maps each word to a count of 1.
6. **reduceByKey**: Aggregates counts by word, summing them up.
7. **collect**: Gathers the final output and prints each word with its count.

Why Use Spark?

Spark is suitable for a wide range of big data applications due to its versatility and speed. It's widely used for:

- **ETL (Extract, Transform, Load)**: Ideal for preprocessing and cleaning large datasets.
- **Data Warehousing and Analytics**: Spark SQL allows SQL-like querying of large datasets.
- **Real-Time Processing**: Spark Streaming enables real-time analytics and decision-making.
- **Machine Learning and AI**: MLlib makes it easy to build scalable machine learning pipelines.

Apache Storm – Stream Processing on top of Hadoop

What is Apache Storm?

- Open-source, distributed **real-time** stream processing system.
- Developed by **Twitter**, now under **Apache Software Foundation**.
- Processes high-velocity data with **low latency** and **high fault tolerance**.

Key Features

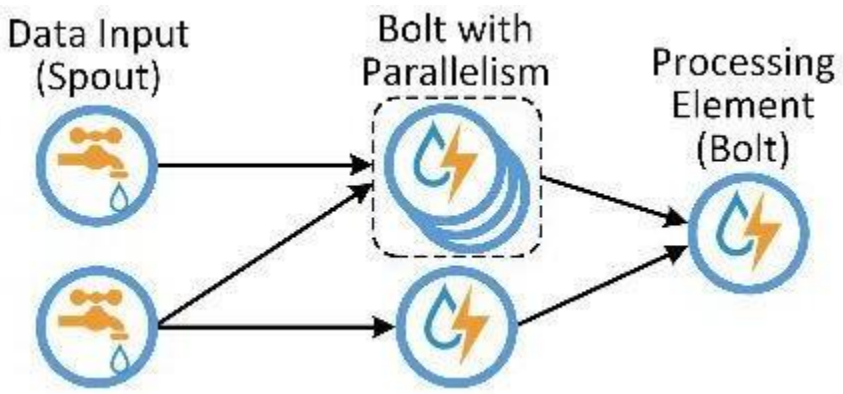
- **Scalable** – Supports distributed processing across multiple nodes.
- **Fault-tolerant** – Automatically reassigns failed tasks.
- **Guaranteed Processing** – Ensures each message is processed at least once.
- **Multi-language Support** – Compatible with Java, Python, and more.

Architecture

- **Spouts** – Data sources (Kafka, databases, APIs).
- **Bolts** – Processing components that filter, aggregate, or store data.
- **Topology** – A directed graph connecting spouts and bolts.

Use Cases

- **Real-time Analytics** – Social media monitoring, fraud detection.
- **Log Processing** – Server logs, IoT data streams.
- **ETL Pipelines** – Continuous data transformation.



Apache Flume

What is Apache Flume?

- Open-source tool for collecting, aggregating, and transporting **large volumes of streaming data** into storage systems like **HDFS**.
- Designed for handling **log data, event streams, and real-time analytics**.
- Developed under the **Apache Software Foundation**.

Key Features

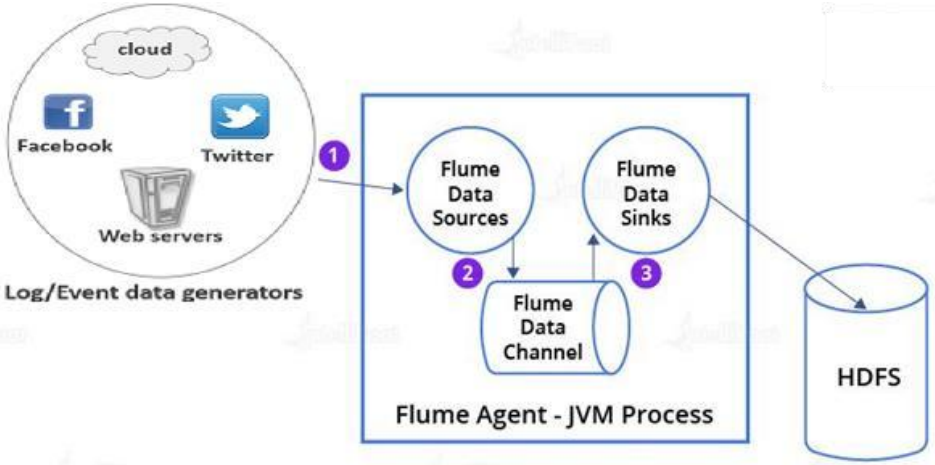
- **Scalable & Reliable** – Handles high-throughput data ingestion.
- **Fault-tolerant** – Ensures no data loss with reliable mechanisms.
- **Flexible & Extensible** – Supports custom data sources and sinks.
- **Streaming Data Support** – Works with logs, events, and real-time sources.

Architecture

- **Source** – Captures data from logs, message queues, or APIs.
- **Channel** – Temporary buffer that holds data (Memory/File-based).
- **Sink** – Sends data to storage (HDFS, HBase, Kafka).

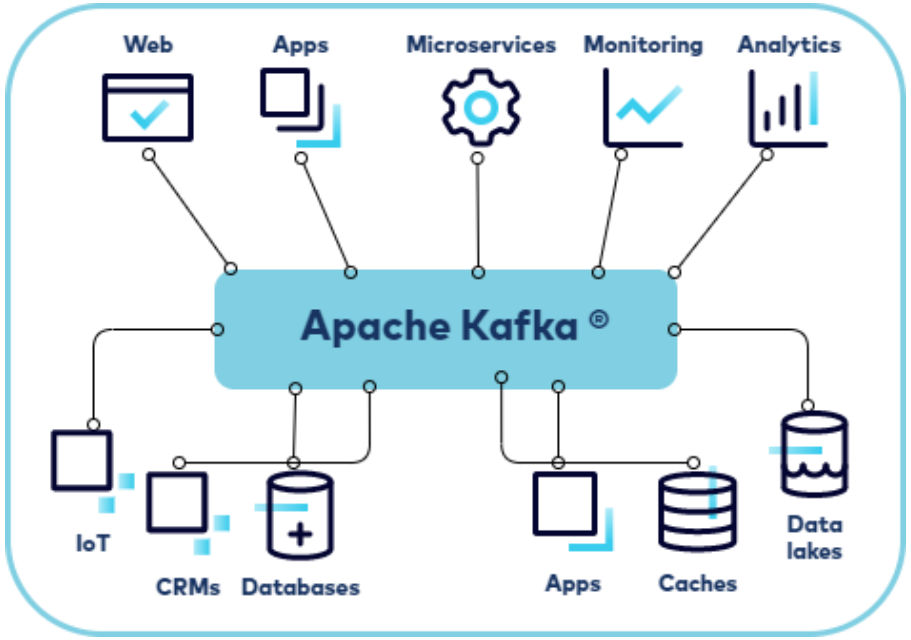
Use Cases

- **Log Collection** – Aggregating server logs for monitoring.
- **Event Streaming** – Collecting user activity data for analytics.
- **ETL Pipelines** – Preprocessing and transporting data to Hadoop ecosystems.



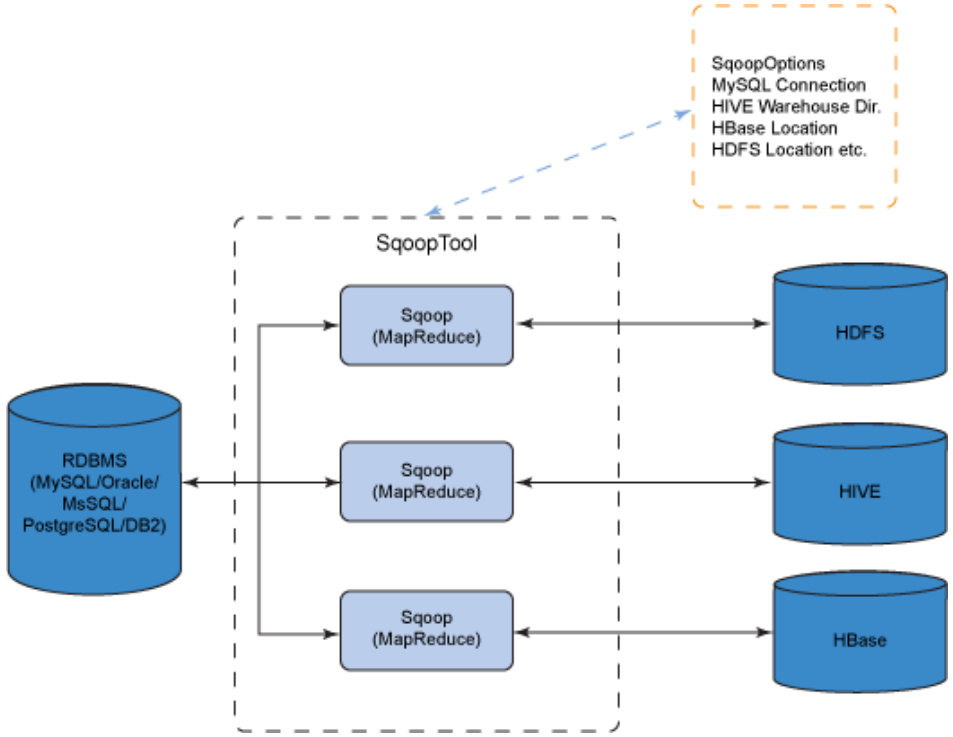
Apache Kafka - Stream Processing on top of Hadoop

- Apache Kafka is a distributed event streaming platform designed to
- handle high-throughput, real-time data feeds.
- Originally developed by LinkedIn, Kafka has become the backbone for data pipelines and real-time analytics in many large-scale systems.
- It serves as a message broker that allows various applications to produce and consume streams of data reliably, efficiently, and at scale.
- **Architecture**:
- **Producer:** An application that sends data to Kafka topics.
- **Consumer:** An application that reads data from Kafka topics.
- **Broker:** A Kafka server that stores data and serves it to consumers.
- Kafka clusters consist of multiple brokers.
- **Topic:** A category or feed name to which records are published. Each topic can have multiple partitions, helping Kafka to parallelize data across the cluster.
- **Use Cases**: Log aggregation, real-time data streams, event sourcing, and data integration.



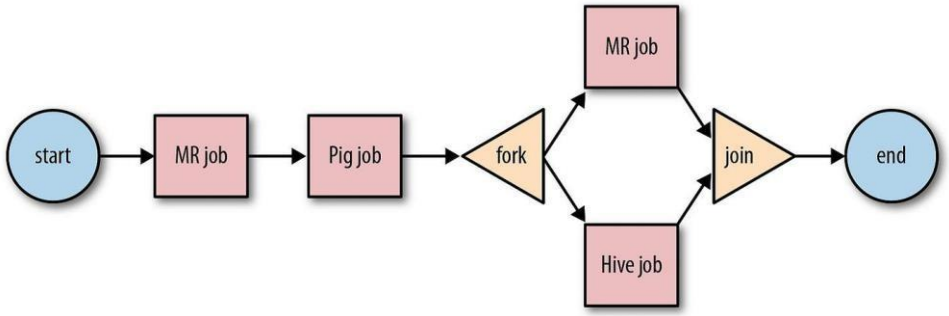
Apache Sqoop

- Apache Sqoop (SQL-to-Hadoop) is a tool designed to efficiently transfer bulk data between Hadoop and structured data stores like relational databases (e.g., MySQL, PostgreSQL, Oracle) and data warehouses.
- Sqoop simplifies the process of importing data from RDBMSs into Hadoop's ecosystem, such as HDFS, Hive, or HBase, and exporting processed data back to these relational databases.
- **Main Functions**:
  - **Import**: Pulls data from relational databases into HDFS.
  - **Export**: Pushes data from HDFS back into relational databases.
- **Typical Use Cases for Sqoop**
- **Data Migration**: Moving historical data from RDBMSs into Hadoop for long-term storage or analysis.
- **ETL Pipelines**: Using Sqoop as part of an ETL pipeline to load data from relational sources into Hadoop for processing and analysis, then exporting results back to the database.
- **Data Synchronization**: Regularly syncing operational data from databases into Hadoop for near real-time analysis.



Apache Oozie

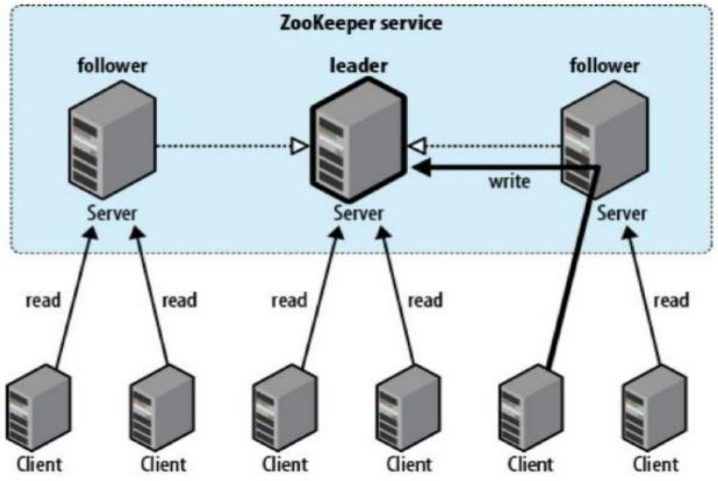
- Apache Oozie is a workflow scheduler system designed to manage and coordinate Hadoop jobs in a sequence or directed acyclic graph (DAG) for end-to-end workflow management.
- Oozie is particularly useful for automating complex data workflows, managing dependencies, and orchestrating multiple jobs across the Hadoop ecosystem (such as MapReduce, Hive, Pig, and Sqoop jobs).
- It supports both time-based (frequency) and data-based triggers,
- making it versatile for different types of big data applications.
- **Main Functions:**
  - **Workflow Management**: Defines and manages job sequences.
  - **Coordination**: Manages job dependencies and schedules based on time and data availability.
  - **Retry Mechanism**: Automatically retries failed jobs.
- **Use Cases**: ETL workflows, batch processing, and managing complex job dependencies.

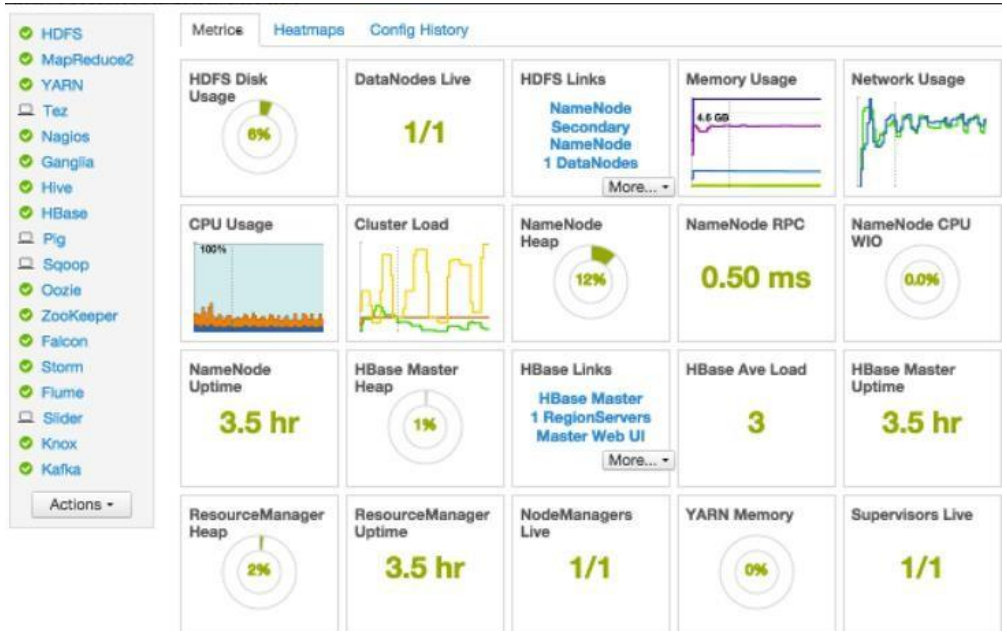
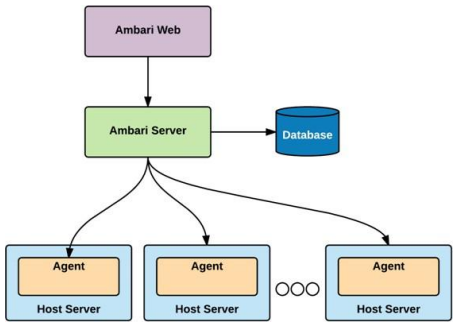


Apache Zookeeper

- Apache ZooKeeper is a centralized service designed for coordinating
- distributed applications.
- ZooKeeper plays a critical role in the Hadoop ecosystem and other distributed systems by helping to maintain the distributed state and providing a reliable way for services to work together.
- **Key Features of Apache ZooKeeper**

1. **Centralized Configuration Management**: ZooKeeper stores configuration data in a centralized manner, allowing distributed applications to access and modify configuration settings easily.
2. **Naming Service**: It provides a naming registry for distributed services, allowing them to find each other and manage resources dynamically.
3. **Synchronization**: ZooKeeper helps synchronize distributed processes, ensuring they work in coordination and without conflicts.
4. **Group Management**: Supports the creation and management of groups, enabling services to join and leave groups dynamically.
5. **Watchers**: ZooKeeper can set watchers on nodes to get notified of changes, allowing applications to react to changes in state.



Apache Ambari

- Apache Ambari is a web-based tool designed for provisioning, managing, and monitoring
- Apache Hadoop clusters.
- It provides a user-friendly interface that simplifies the management of complex Hadoop environments, making it easier for administrators and operators to deploy and manage various components of the Hadoop ecosystem.
- **Key Features of Apache Ambari**
- **Cluster Provisioning**: Ambari simplifies the installation and configuration of Hadoop clusters, allowing users to easily add or remove nodes and services through a guided installation wizard.
- **Service Management**: Users can start, stop, and restart various Hadoop services (such as HDFS, YARN, Hive, HBase, etc.) through the Ambari interface.
- **Monitoring and Alerts**: Ambari provides dashboards for real-time monitoring of cluster health, service metrics, and resource usage. It also supports setting up alerts for various cluster events or metrics.
- **Configuration Management**: It allows for easy management of configuration settings for Hadoop services, including the ability to apply configurations in a centralized manner.
- **User Management**: Administrators can manage users and their permissions, ensuring secure access to cluster resources and configurations.
- **Integration with Ambari Metrics and Alerts**: Supports the collection and visualization of metrics from different Hadoop components, allowing for comprehensive monitoring and alerting.